

Document Title	Explanation of Adaptive Platform Software Architecture
Document Owner	AUTOSAR
Document Responsibility	AUTOSAR
Document Identification No	982

Document Status	published
Part of AUTOSAR Standard	Adaptive Platform
Part of Standard Release	R22-11

Document Change History			
Date	Release	Changed by	Description
2022-11-24	R22-11	AUTOSAR Release Management	- Added an initial version of the Use Case View that shows the main use cases for the Functional Clusters of the AUTOSAR Adaptive Platform. - Reworked the Runtime View to show a proposed internal realization of the use cases in the Use Case View by means of interaction diagrams. - Added functional cluster Firewall
2021-11-25	R21-11	AUTOSAR Release Management	- Applied a more fine-grained description schema for functional clusters and interfaces in the Building Block View. - Removed functional cluster RESTful Communication - Added functional cluster Adaptive Intrusion Detection System Manager - Added section for clarification of diagnostic deployment options
2020-11-30	R20-11	AUTOSAR Release Management	Initial release

Disclaimer

This work (specification and/or software implementation) and the material contained in it, as released by AUTOSAR, is for the purpose of information only. AUTOSAR and the companies that have contributed to it shall not be liable for any use of the work.

The material contained in this work is protected by copyright and other types of intellectual property rights. The commercial exploitation of the material contained in this work requires a license to such intellectual property rights.

This work may be utilized or reproduced without any modification, in any form or by any means, for informational purposes only. For any other purpose, no part of the work may be utilized or reproduced, in any form or by any means, without permission in writing from the publisher.

The work has been developed for automotive applications only. It has neither been developed, nor tested for non-automotive applications.

The word AUTOSAR and the AUTOSAR logo are registered trademarks.

Contents

1	Introduction	6
1.1	Objectives	6
1.2	Scope	7
1.3	Document Structure	7
2	Definition of Terms and Acronyms	8
2.1	Acronyms and Abbreviations	8
2.2	Definition of Terms	8
3	Related Documentation	9
4	Overview and Goals	10
4.1	Requirements Overview	10
4.2	Quality Goals	12
4.3	Stakeholders	12
5	Architecture Constraints	13
5.1	Internal Interfaces	13
5.2	Distributed Work	14
6	Quality Requirements	15
6.1	Quality Attributes	15
6.1.1	AUTOSAR Adaptive Platform Standard	15
6.1.2	AUTOSAR Adaptive Platform Stack	17
6.1.3	AUTOSAR Adaptive Application	18
6.2	Quality Scenarios	19
7	System Scope and Context	20
7.1	Adaptive Application	20
7.2	Dependencies	20
7.2.1	Crypto Provider	20
7.2.2	Operating System	21
7.2.3	Watchdog	21
7.3	External Systems	21
7.3.1	AUTOSAR Adaptive Application	21
7.3.2	AUTOSAR Classic Platform	22
7.3.3	Third-party Platform	22
7.3.4	Diagnostic Client	22
7.3.5	Backend	22
8	Solution Strategy	23
8.1	Architectural Approach	23
8.2	Decomposition Strategy	23
8.3	UML Profile	24
8.4	Technology	25

8.4.1	Implementation Language	25
8.4.2	Parallel Processing	25
8.5	Design Principles	26
8.5.1	Leveraging existing standards	26
8.5.2	SOLID principles	26
8.5.3	Acyclic Dependencies Principle	27
8.6	Deployment	27
8.7	Verification and Validation	28
9	Building Block View	29
9.1	Overview	29
9.1.1	Description pattern	29
9.2	Runtime	30
9.2.1	Execution Management	30
9.2.2	State Management	36
9.2.3	Log and Trace	42
9.2.4	Core	46
9.2.5	Operating System Interface	47
9.3	Communication	49
9.3.1	Communication Management	49
9.3.2	Network Management	54
9.3.3	Time Synchronization	56
9.4	Storage	62
9.4.1	Persistency	63
9.5	Security	70
9.5.1	Cryptography	70
9.5.2	Identity and Access Management	94
9.5.3	Adaptive Intrusion Detection System Manager	96
9.5.4	Firewall	99
9.6	Safety	101
9.6.1	Platform Health Management	101
9.7	Configuration	106
9.7.1	Update and Configuration Management	106
9.7.2	Registry	114
9.8	Diagnostics	116
9.8.1	Diagnostic Management	117
10	Use-Case View	134
10.1	Runtime	135
10.1.1	Execution Management	135
10.1.2	State Management	137
10.2	Storage	139
10.2.1	Persistency	139
10.3	Security	145
10.3.1	Firewall	145
11	Runtime View	147

11.1	Runtime	147
11.1.1	Execution Management	147
11.1.2	State Management	152
11.2	Storage	154
11.2.1	Persistency	154
11.3	Security	158
11.3.1	Firewall	158
12	Deployment View	160
12.1	Vehicle Software Deployment	160
12.2	Deployment of Software Packages on a Machine	161
13	Cross-cutting Concepts	163
13.1	Overview of Platform Entities	163
13.2	Function Group	164
13.3	Function Group State	164
13.4	Software Cluster	164
13.5	Machine	167
13.6	Manifest	167
13.7	Application Design	168
13.8	Execution Manifest	169
13.9	Service Instance Manifest	169
13.10	Machine Manifest	170
13.11	Diagnostics deployment	170
13.12	Error Handling	172
13.13	Trusted Platform	172
13.14	Secure Communication	173
14	Risks and Technical Debt	174
14.1	Risks	174
14.1.1	Risk Assessment	174
14.1.2	Risk List	175
14.2	Technical Debt	175

1 Introduction

This explanatory document provides detailed technical description of the software architecture of the AUTOSAR Adaptive Platform standard with the main focus on the architecture model.

1.1 Objectives

This document is an architecture description of the AUTOSAR Adaptive Platform in accordance to [1, ISO/IEC 42010] and has the following main objectives:

- Identify the **stakeholders** of the AUTOSAR Adaptive Platform and their **concerns**.
- Identify the **system scope** and provide **overview information** of the AUTOSAR Adaptive Platform.
- Provide definitions for all used **architecture viewpoints** and a **mapping of all stakeholder concerns to those viewpoints**.
- Provide an **architecture view** and its **architecture models** for each architecture viewpoint used in this architecture description.
- Provide **correspondence rules** and **correspondences** among the contents of this architecture description.
- Provide an **architecture rationale** (explanation, justification, reasoning for decisions made) on a high level. A more in-depth documentation of decisions is provided in [2, EXP_SWArchitecturalDecisions].
- Provide a **record of known inconsistencies and gaps** among the architecture description.

There is some potential for ambiguity about the term "architecture". Association with this term is quite different e.g., for a mass production project in contrast with Adaptive Platform standardization. For system development of an automotive embedded computer the software architecture usually defines the details of the structural and the behavioral architecture views down to module level. In contrast the architecture of AUTOSAR Adaptive Platform lacks such details deliberately to provide more degrees of freedom for stack vendors in their solution design.

Beyond the specification of APIs the term "architecture" for Adaptive Platform refers to guidelines how to apply the standard to concrete development projects.

This document describes the original architectural design of the AUTOSAR Adaptive Platform including details how the building blocks should interact with each other. It is an example how an implementation of the standard should work internally. However, a stack vendor is free to choose another design as long as it complies with the binding AUTOSAR Adaptive Platform standard.

4 Overview and Goals

In conventional automotive systems ECUs are used to replace or augment electro-mechanical systems. Those resource constrained, deeply-embedded ECUs typically perform basic control functions by creating electrical output signals (e.g. for actors) based on input signals (e.g. from sensors) and information from other ECUs connected to the vehicle network. Much of the control software is specifically designed and implemented for the target vehicle and does not change significantly during vehicle lifetime. The AUTOSAR Classic Platform standard addresses the needs of these deeply-embedded systems.

Recent and future vehicle functions, such as highly automated driving, will introduce complex and computing resource demanding software that shall fulfill strict safety, integrity and security requirements. Such software performs for example, environment perception and behavior planning, and interacts with external backend and infrastructure systems. The software in the vehicle regularly needs to be updated during the life-cycle of the vehicle, due to evolving external systems, improved or added functionality, or security problems. The AUTOSAR Classic Platform standard cannot fulfill the needs of such systems. Therefore, AUTOSAR specifies a second software platform, the AUTOSAR Adaptive Platform. It provides high-performance computing and communication mechanisms as well as a flexible software configuration, for example, to support software update over-the-air. Features that are specifically defined for the AUTOSAR Classic Platform, such as access to electrical signals and automotive specific bus systems, can be integrated into the AUTOSAR Adaptive Platform but is not in the focus of standardization.

4.1 Requirements Overview

This section provides an overview of the basic requirements for the AUTOSAR Adaptive Platform that impact its architecture. The corresponding requirement identifiers are provided in square brackets. Please refer to [4, RS_Main] and [5, RS_General] for any details, rationale or intended use-cases of these requirements.

Support of state-of-the-art Technology

The AUTOSAR Adaptive Platform aims to support resource-intensive (memory, cpu) applications on state-of-the-art hardware. Therefore, the AUTOSAR Adaptive Platform shall support high performance computing platforms [RS_Main_00002] as well as virtualized environments [RS_Main_00511]. The AUTOSAR Adaptive Platform shall be able to run multiple applications in parallel [RS_Main_00049], each with concurrent application internal control flows [RS_Main_00050].

7 System Scope and Context

This chapter provides an overview of the AUTOSAR Adaptive Platform system context by separating the AUTOSAR Adaptive Platform and its communication partners (e.g., external systems). Considering Figure 7.1, there are three categories of communication partners for the AUTOSAR Adaptive Platform.

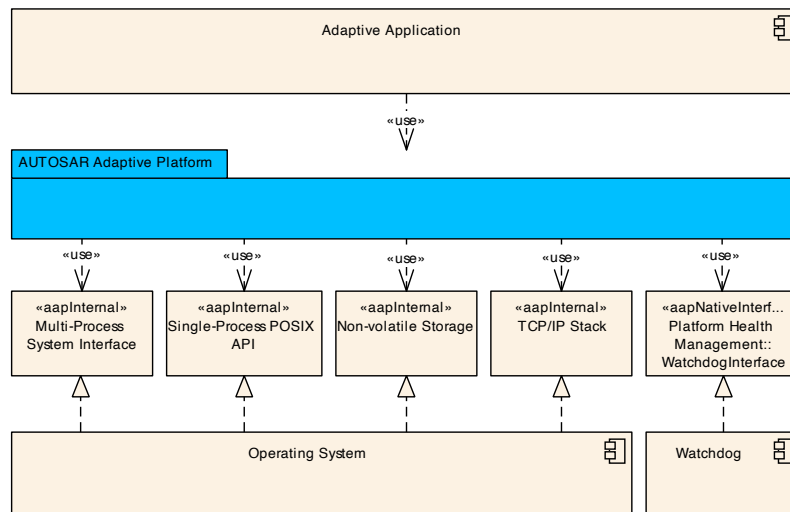


Figure 7.1: Overview of AUTOSAR Adaptive Platform and its context

The AUTOSAR Adaptive Platform is conceptually a middleware. AUTOSAR Adaptive Platform provides services to Adaptive Applications beyond those available from the underlying operating system, drivers, and extensions (cf. Section 7.2). Section 7.3 describes the third category that are external systems communicating with (an Adaptive Application via) AUTOSAR Adaptive Platform.

7.1 Adaptive Application

Adaptive Applications are built on the functionality provided by the AUTOSAR Adaptive Platform. They directly use the various interfaces provided by the individual building blocks of AUTOSAR Adaptive Platform described in more detail in chapter 9.

7.2 Dependencies

7.2.1 Crypto Provider

Crypto Provider is a component that provides implementations of cryptographic routines and hash functions to the AUTOSAR Adaptive Platform.

7.2.2 Operating System

The [Operating System](#) is the main component that AUTOSAR Adaptive Platform uses to provide its services. The [Operating System](#) controls processes and threads, and provides inter-process communication facilities. The [Operating System](#) also provides access to network interfaces, protocols like TCP/IP, and access to non-volatile storage.

7.2.3 Watchdog

The [Watchdog](#) is a component to control the hardware watchdog of the machine an AUTOSAR Adaptive Platform runs on.

7.3 External Systems

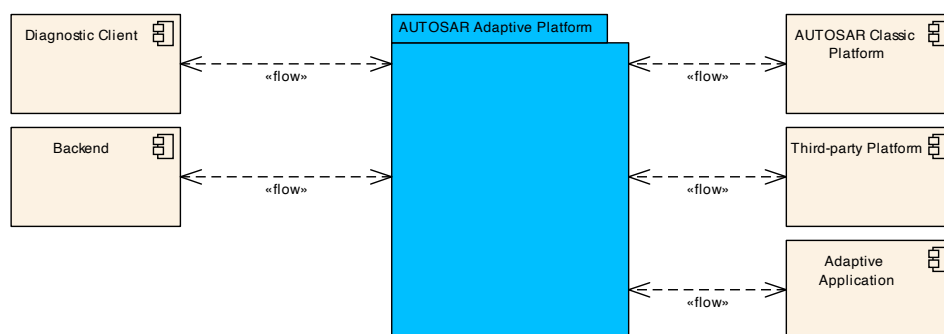


Figure 7.2: Overview of the AUTOSAR Adaptive Platform and external systems

The AUTOSAR Adaptive Platform supports applications that are operated in heterogeneous environments. This section lists the external systems that AUTOSAR Adaptive Platform is intended to interface.

7.3.1 AUTOSAR Adaptive Application

There may be many [Adaptive Applications](#) deployed in a vehicle on different Machines. An [Adaptive Application](#) that does not run on the current instance of the AUTOSAR Adaptive Platform is therefore an external system to the AUTOSAR Adaptive Platform. Such [Adaptive Applications](#) may exchange data such as sensor or status information by means of `ara::com` service interfaces.

7.3.2 AUTOSAR Classic Platform

The [AUTOSAR Classic Platform](#) is the main platform for deeply-embedded applications such as sensor/actor systems. [Adaptive Applications](#) may require access for example to sensor data provided by an [AUTOSAR Classic Platform](#) ECU and vice versa.

7.3.3 Third-party Platform

Besides the both platforms ([AUTOSAR Adaptive Platform](#) and [AUTOSAR Classic Platform](#)) provided by AUTOSAR, there may be ECUs in a vehicle and other systems that are built on different platforms that need to communicate with an [Adaptive Application](#) via [AUTOSAR Adaptive Platform](#).

7.3.4 Diagnostic Client

A [Diagnostic Client](#) uses the diagnostic services provided by the [AUTOSAR Adaptive Platform](#).

7.3.5 Backend

A [Backend](#) system provides [Software Packages](#) for download and controls the update process via [Update and Configuration Management](#).

8 Solution Strategy

The AUTOSAR Adaptive Platform is a standard for an automotive middleware. It is not a concrete implementation. The AUTOSAR Adaptive Platform standard leaves a certain degree of freedom to its implementers by defining requirements and software specifications that need to be fulfilled without specifying how.

8.1 Architectural Approach

To support the complex applications, while allowing maximum flexibility and scalability in processing distribution and compute resource allocations, AUTOSAR Adaptive Platform follows the concept of a **service-oriented architecture (SOA)**. In a service-oriented architecture a system consists of a set of services, in which one may use another in turn, and applications that use one or more of the services depending on its needs. Often service-oriented architectures exhibit system-of-system characteristics, which AUTOSAR Adaptive Platform also has. A service, for instance, may reside on a local ECU that an application also runs, or it can be on a remote ECU, which is also running another instance of AP. The application code is the same in both cases - the communication infrastructure will take care of the difference providing transparent communication. Another way to look at this architecture is that of distributed computing, communicating over some form of message passing. At large, all these represent the same concept. This message passing, communication-based architecture can also benefit from the rise of fast and high-bandwidth communication such as Ethernet.

8.2 Decomposition Strategy

The building blocks of the AUTOSAR Adaptive Platform architecture are refined step-by-step in this document according to the model depicted in figure 8.1. The top-level categories are chosen to give an overview from a users perspective what kind of functionality the AUTOSAR Adaptive Platform provides. A category contains one or more **Functional Clusters**. The **Functional Clusters** of the AUTOSAR Adaptive Platform are defined to group a specific coherent technical functionality. **Functional Clusters** themselves specify a set of interfaces and components to provide and realize that technical functionality. The building block view also contains information of the **Functional Clusters** interdependencies based on interfaces from other **Functional Clusters** they use. However, note that these interdependencies are recommendations rather than strict specifications because they would constrain implementations.

Stereotype	Metaclass	Semantics
aapPortInterface	Interface	An aapPortInterface relates to an element in the Manifest using the Instance-Specifier pattern. Such interfaces are either defined in the respective programming language of the stack implementation or they are generated in that language by the stack tooling. The property Generated specifies if they are generated.
aapAraComServiceInterface	Interface	An ara::com service interface defined and configured in the Manifest. The tooling of the stack implementation will generate stub and skeleton implementations of these interfaces in the respective programming language.
aapFunctionalCluster	Component	A functional cluster of the AUTOSAR Adaptive Platform.
aapServiceMethod	Operation	A method specified as part of an ara::com service interface.
aapCallbackMethod	Operation	A method that acts as a callback.
aapServiceField	Attribute	A field specified as part of an ara::com service interface.
aapServiceEvent	Attribute	An event specified as part of an ara::com service interface.
aapProvidedPort	Usage, Realization	Denotes that the underlying relationship is configured in the Manifest using a provided port.
aapRequiredPort	Usage, Realization	Denotes that the underlying relationship is configured in the Manifest using a required port.

Table 8.1: Overview of Stereotypes

8.4 Technology

8.4.1 Implementation Language

C++ is the programming language of choice for the AUTOSAR Adaptive Platform and [Adaptive Applications](#). C++ was chosen due to its safer programming model (compared to C) and availability of certified compilers that produce highly optimized machine code. Such properties are especially important for safety- and performance-critical, real-time applications (such as typical [Adaptive Applications](#)) where C++ has become more and more popular in the software industry and academics.

8.4.2 Parallel Processing

Although the design for AUTOSAR Adaptive Platform as a service-oriented architecture inherently leverages parallel processing, the advancement of (heterogeneous) many-

core processors offers additional opportunities. The AUTOSAR Adaptive Platform is designed to scale its functionality and performance as (heterogeneous) many-core technologies advance. Hardware and platform interface specifications are one part of that equation. However, advancements in operating system and hypervisor technologies as well as development tools (for example automatic parallelization) are also crucial and are to be fulfilled by AUTOSAR Adaptive Platform providers, the software industry, and academics.

8.5 Design Principles

The architecture of the AUTOSAR Adaptive Platform is based on several design principles that are outlined below.

8.5.1 Leveraging existing standards

AUTOSAR Adaptive Platform aims to leverage existing standards and specifications wherever possible. For example, AUTOSAR Adaptive Platform is built on the existing and open C++ standard (cf. Section 8.4.1) to facilitate a faster development of the AUTOSAR Adaptive Platform itself and benefiting from the eco-systems of such standards. It is, therefore, a critical focus in developing the AUTOSAR Adaptive Platform specification not to casually introduce a new replacement functionality that an existing standard already offers. For instance, no new interfaces are casually introduced just because an existing standard provides the functionality required but the interface is superficially hard to understand.

8.5.2 SOLID principles

The SOLID principles [7] are a central part of the design principles of AUTOSAR. While these five principles are all valid, only the Single-responsibility Principle, the Interface Segregation Principle and the Dependency Inversion Principle are relevant on the abstraction level of this document. Therefore, they are elaborated in the following.

8.5.2.1 Single-responsibility Principle

The single-responsibility principle (SRP, SWEBOOK3) [7] states that a component or class should be responsible for a single part of the overall functionality provided by the software. That responsibility should be encapsulated by the component or class. The services provided by the component or class (via its interface(s)) should be closely aligned with its responsibility.

The single-responsibility principle minimizes the reasons (i.e. a change to the single responsibility) that require a change to its interface. Thus, it minimizes impact on clients of such an interface and leads to a more maintainable architecture (or code).

8.5.2.2 Interface Segregation Principle

The interface segregation principle (ISP) [7], [8] states that clients should not be forced to depend on methods that they don't use. As a consequence of the interface segregation principle, interfaces should be split up to reflect different roles of clients.

Similar to the single-responsibility principle, the segregation of interfaces reduce the impact of a change to an interface to the clients and suppliers of an segregated interface.

8.5.2.3 Dependency Inversion Principle

The dependency inversion principle (DIP) [7], [8] states that high-level building blocks should not depend on low-level building blocks. Both should depend on abstractions (e.g. interfaces). Furthermore, the dependency inversion principle states that abstractions (e.g. interfaces) should not depend on details. Details (e.g. a concrete implementation) should depend on abstractions.

The dependency inversion principle results in a decoupling of the implementations of building blocks. This is important to scale implementation efforts (cf. Section 5.2) and to perform proper integration tests.

8.5.3 Acyclic Dependencies Principle

The acyclic dependencies principle (ADP) [7], [8] states that dependencies between building blocks should form a directed acyclic graph.

The acyclic dependencies principle helps to identify participating building blocks and reason about error propagation and freedom from interference. In general, it also reduces the extend of building blocks to consider during activities such as test, build and deployment.

8.6 Deployment

The AUTOSAR Adaptive Platform supports the incremental deployment of applications, where resources and communications are managed dynamically to reduce the effort for software development and integration, enabling short iteration cycles. Incremental deployment also supports explorative software development phases. For product delivery, the AUTOSAR Adaptive Platform allows the system integrator to carefully

limit dynamic behavior to reduce the risk of unwanted or adverse effects allowing safety qualification. Dynamic behavior of an application will be limited by constraints stated in the `Execution Manifest` (cf. Section 13.8), for example, dynamic allocation of resources and communication paths are only possible in defined ways, within configured ranges. Implementations of an AUTOSAR Adaptive Platform may further remove dynamic capabilities from the software configuration for production use. Examples of reduced behavioral dynamics might be:

- Pre-determination of the service discovery process
- Restriction of dynamic memory allocation to the startup phase only
- Fair scheduling policy in addition to priority-based scheduling
- Fixed allocation of processes to CPU cores
- Access to pre-existing files in the file-system only
- Constraints for AUTOSAR Adaptive Platform API usage by applications
- Execution of authenticated code only

8.7 Verification and Validation

The AUTOSAR Adaptive Platform standard uses a dedicated implementation of the standard (AUTOSAR Adaptive Platform Demonstrator) to validate the requirements and to verify the (still abstract) software design imposed by the individual software specifications.

9 Building Block View

This chapter provides an overview of the static structure of the AUTOSAR Adaptive Platform by describing the high-level building blocks and their inter-dependencies. Please note that the use of interfaces between `FunctionalClusters` in the AUTOSAR Adaptive Platform is currently not standardized. Some aspects, for example, access management, are also not yet fully incorporated and standardized in all `FunctionalClusters`.

9.1 Overview

Figure 9.1 provides an overview of the different categories of building blocks available in the AUTOSAR Adaptive Platform. The categories are explained in more detail in the subsequent sections.

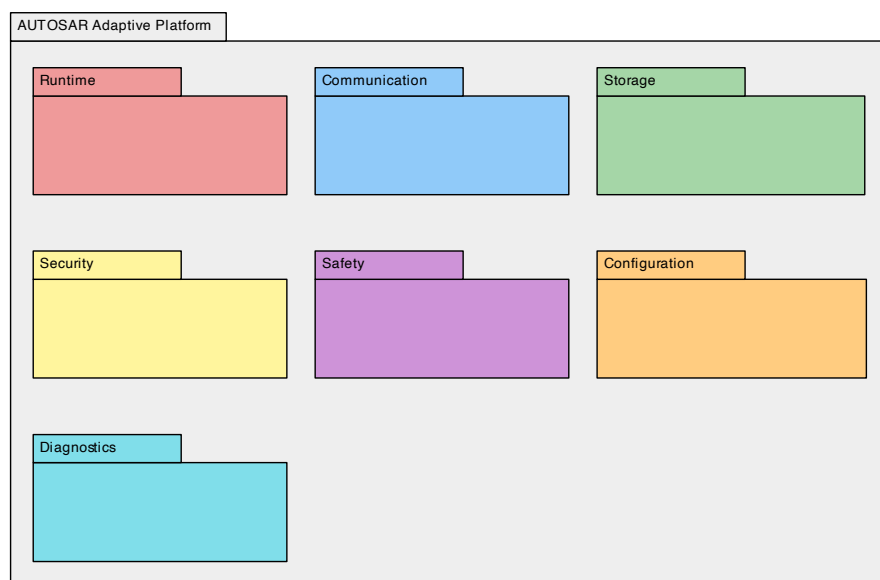


Figure 9.1: Overview of the AUTOSAR Adaptive Platform and its building block categories

9.1.1 Description pattern

The description of all building blocks (`FunctionalClusters`) in this section uses the same pattern. Each `FunctionalCluster` is described in a separate section of the document. Such a section starts with an overview of the responsibilities of the `FunctionalCluster` followed by a sub-section called "Defined interfaces". The sub-section "Defined interfaces" lists all architectural interfaces specified in the namespace of the `FunctionalCluster`. Each interface is detailed in a separate table.

The subsection "Provided interfaces" then lists all interfaces provided by the `FunctionalCluster` (i.e. it is fully implemented by the `FunctionalCluster`) to other `FunctionalCluster`s.

The last subsection "Required interfaces" lists all interfaces required by the `FunctionalCluster` from other `FunctionalCluster`s and external components like the operating system.

9.2 Runtime

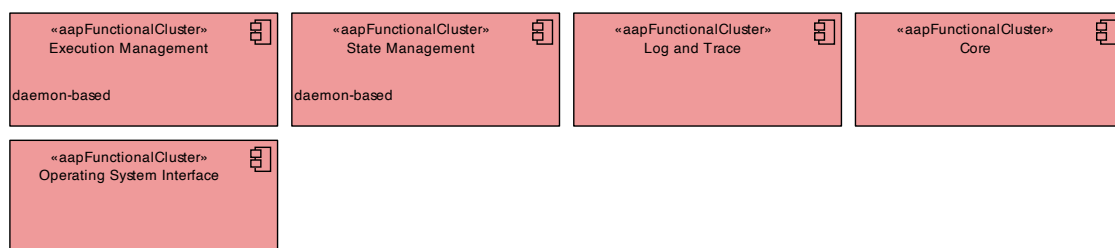


Figure 9.2: Functional Clusters in category Runtime

9.2.1 Execution Management

Name:	Execution Management
Short Name:	exec
Category:	Runtime
Daemon-based:	Yes
Responsibilities:	<code>Execution Management</code> is responsible to control <code>Processes</code> of the AUTOSAR Adaptive Platform and <code>Adaptive Applications</code>. That is, it starts, configures, and stops <code>Processes</code> as configured in <code>Function Group States</code> using interfaces of the <code>Operating System</code>. The <code>Operating System</code> is responsible for runtime scheduling of those <code>Processes</code>. The configuration of <code>Processes</code> that <code>Execution Management</code> performs includes limiting their resource consumption (CPU time, memory) using <code>Resource Groups</code> provided by the <code>Operating System</code>. <code>Execution Management</code> is the entry point of AUTOSAR Adaptive Platform and is started by the <code>Operating System</code> during system boot. <code>Execution Management</code> then controls the startup and shutdown of the AUTOSAR Adaptive Platform (see use cases <code>Start Adaptive Platform</code> and <code>Shutdown Adaptive Platform</code> for details). <code>Execution Management</code> optionally supports authenticated startup where it maintains the chain of trust when starting from a <code>Trust Anchor</code>. During authenticated startup <code>Execution Management</code> validates the authenticity and integrity of <code>Processes</code> and shall prevent their execution if violations are detected. Through these mechanisms, a trusted platform can be established.

9.2.5.3 Required interfaces

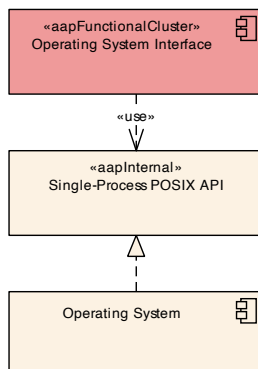


Figure 9.22: Interfaces required by Operating System Interface

Interface	Purpose
Single-Process POSIX API	Operating System Interface uses this interface to provide the functionality of the POSIX PSE51 profile, e.g. Threads . Usually the whole POSIX PSE51 profile API will already be provided by the underlying operating system. In this case, the Operating System Interface will not have an implementation in the Adaptive Platform stack.

Table 9.8: Interfaces required by Operating System Interface

9.3 Communication

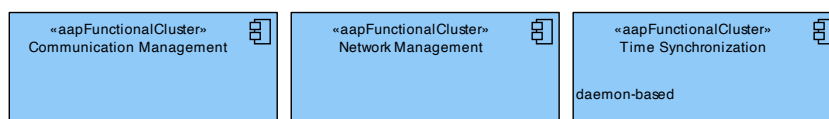


Figure 9.23: Functional Clusters in category Communication

9.3.1 Communication Management

Name:	Communication Management
Short Name:	com
Category:	Communication
Daemon-based:	No





Responsibilities:	Communication Management is responsible for all levels of service-oriented and raw communication between applications in a distributed real-time embedded environment. That is, intra-process communication, inter-process communication and inter-machine communication. The latter is also possible with AUTOSAR Classic Platforms and third-party platforms. Communication paths can be established at design-, start-up-, and run-time. Communication Management consists of a generic part that handles brokering and configuration as well as generated skeletons for service providers and respective proxies for service consumers.
--------------------------	---

9.3.1.1 Defined interfaces

The interfaces of **Communication Management** are categorized into interfaces for raw data streams (see Section 9.3.1.1.1), interfaces for SecOC (see Section 9.3.1.1.2), and interfaces freshness value management (see Section 9.3.1.1.3). Please note that a implementation of **Communication Management** will generate additional interfaces for each modeled Service, e.g. a Proxy and a Skeleton interface. However, these generated interfaces are not covered in this document.

9.3.1.1.1 Interfaces for raw data streams

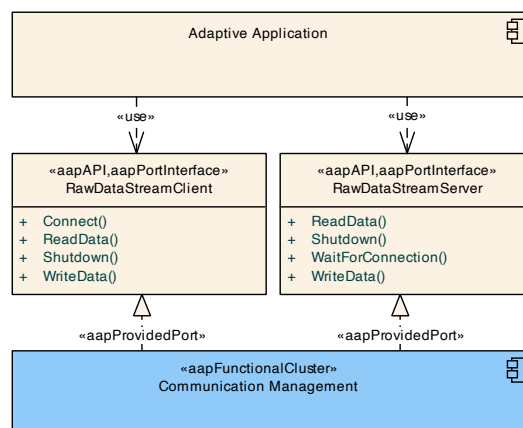


Figure 9.24: Interfaces for raw data streams

Name:	RawDataStreamClient
Technology:	Port interface
Generated:	No
Meta-model interface type:	RawDataStreamClientInterface
Usage:	Public API
Description:	This interface provides functionality for a client reading and writing binary data streams over a network connection.





Operations:	Connect	Sets up a socket connection for the raw data stream defined by the instance, and establish a connection to the TCP server.
	ReadData	Requests to read a number of bytes of data from the socket connection for the raw data stream defined by the instance.
	Shutdown	Closes the socket connection for the raw data stream defined by the instance. Both, the receiving and the sending part of the socket connection shall be shut down.
	WriteData	Requests to write of a number of bytes to the socket connection for the raw data stream defined by the instance.

Name:	RawDataStreamServer	
Technology:	Port interface	
Generated:	No	
Meta-model interface type:	RawDataStreamServerInterface	
Usage:	Public API	
Description:	This interface provides functionality for a server reading and writing binary data streams over a network connection.	
Operations:	ReadData	Requests to read a number of bytes of data from the socket connection for the raw data stream defined by the instance.
	Shutdown	Closes the socket connection for the raw data stream defined by the instance. Both the receiving and the sending part of the socket connection shall be shut down.
	WaitForConnection	Initializes the socket and prepare the RawDataStreamServer instance for incoming connections.
	WriteData	Requests to write of a number of bytes to the socket connection for the raw data stream defined by the instance.

9.3.1.1.2 Interfaces for SecOC

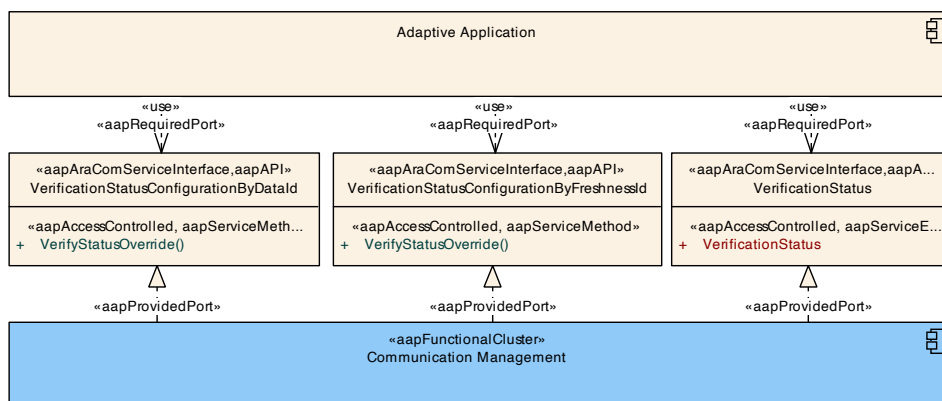


Figure 9.25: Interfaces for SecOC

Name:	VerificationStatus	
Technology:	ara::com service interface	
Usage:	Public API	
Description:	This interface provides an event to informed about the verification status of messages.	
Events:	VerificationStatus	This event is fired for each verification and holds the verification status.

Name:	VerificationStatusConfigurationByDataId	
Technology:	ara::com service interface	
Usage:	Public API	
Description:	This interface provides functionality to control the verification status of messages.	
Operations:	VerifyStatusOverride	This service method provides the ability to force to accept or to drop a message with or without performing the verification of authenticator or independent of the authenticator verification result, and to force a specific result allowing additional fault handling in the application.

Name:	VerificationStatusConfigurationByFreshnessId	
Technology:	ara::com service interface	
Usage:	Public API	
Description:	This interface provides functionality to control the verification status of messages.	
Operations:	VerifyStatusOverride	This service method provides the ability to force to accept or to drop a message with or without performing the verification of authenticator or independent of the authenticator verification result, and to force a specific result allowing additional fault handling in the application.

9.3.1.1.3 Interfaces for freshness value management

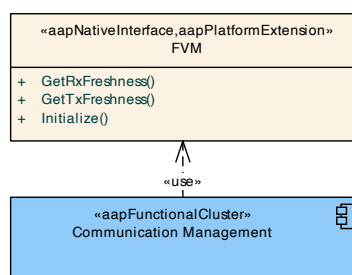


Figure 9.26: Interfaces for freshness value management

Name:	FVM
Technology:	Native interface
Usage:	Platform extension





Description:	This interface provides functionality for freshness value management.	
Operations:	GetRxFreshness	Obtain the current freshness value for received messages.
	GetTxFreshness	Obtain the current freshness value for transmitted messages.
	Initialize	Initializes freshness value manager plugin implementation.

9.3.1.2 Provided interfaces

Communication Management currently provides no interfaces to other Functional Clusters.

9.3.1.3 Required interfaces

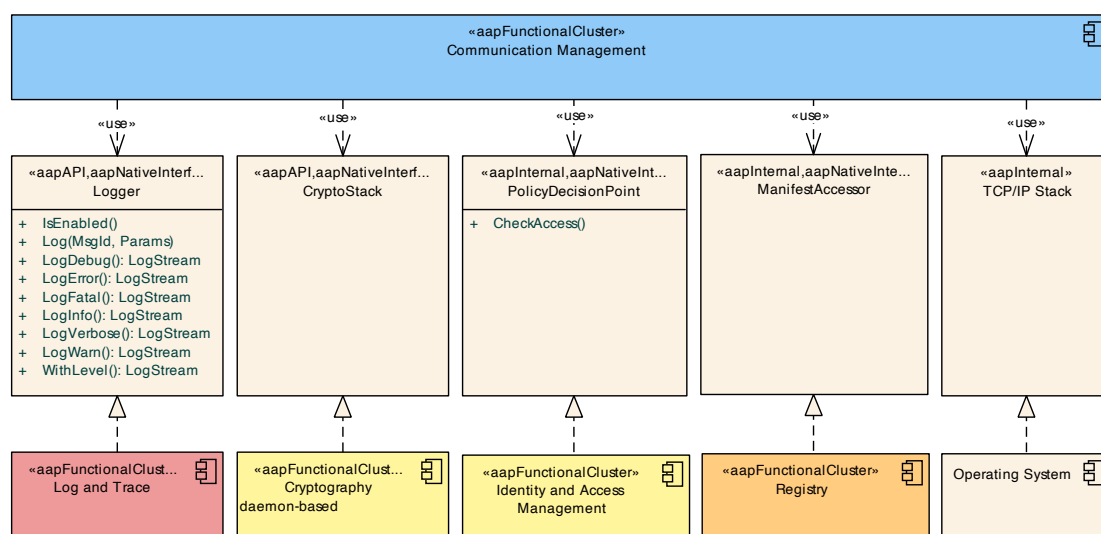


Figure 9.27: Interfaces required by Communication Management

Interface	Purpose
TCP/IP Stack	Communication Management shall use this interface to establish and control TCP/IP-based network connections.
Adaptive Intrusion Detection System Manager::EventReporter	Communication Management shall use this interface to report security events.
Communication Management::FVM	Communication Management shall use this interface to get freshness values.
Cryptography::CryptoStack	Communication Management shall use this interface to establish encrypted connections and generate / verify checksums (MAC).
Identity and Access Management::Policy DecisionPoint	Communication Management shall use this interface to check access to ara::com service methods, fields, and events.





Interface	Purpose
Log and Trace::Logger	Communication Management shall use this interface to log standardized messages.
Operating System Interface::OperatingSystem Interface	Communication Management should use this interface to create and control Threads used by the implementation.
Registry::ManifestAccessor	Communication Management shall use this interface to read service information from the Manifests.

Table 9.9: Interfaces required by Communication Management

9.3.2 Network Management

Name:	Network Management
Short Name:	nm
Category:	Communication
Daemon-based:	No
Responsibilities:	Network Management provides functionality to request and query the network states for logical network handles, Such network handles can be mapped to physical or partial networks.

9.3.2.1 Defined interfaces

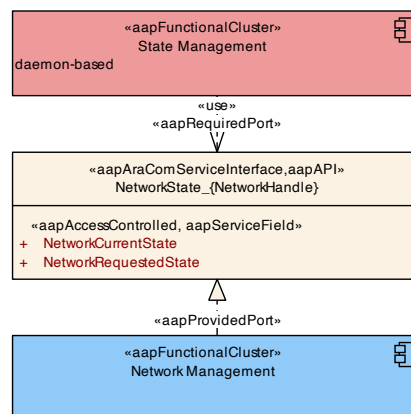


Figure 9.28: Interfaces for time base providers

Name:	NetworkState_{NetworkHandle}
Technology:	ara::com service interface
Usage:	Public API
Description:	This interface provides information about network status per NetworkHandle. This interface is intended to be used by State Management only.





Fields:	NetworkCurrentState	Indicates if a PNC / VLAN / Physical Network is currently active or not.
	NetworkRequestedState	Request a PNC / VLAN / Physical Network to get active or to release it.

9.3.2.2 Provided interfaces

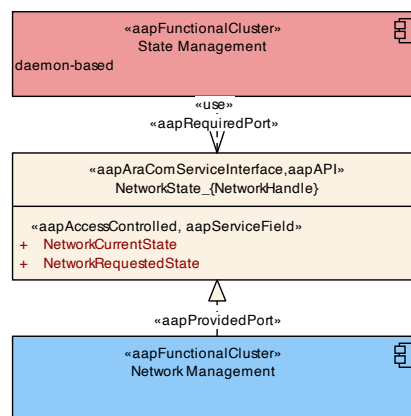


Figure 9.29: Users of Network Management interfaces

Interface	Requiring functional clusters
Network Management::NetworkState_{Network Handle}	State Management

Table 9.10: Interfaces provided by Network Management to other Functional Clusters

9.3.2.3 Required interfaces

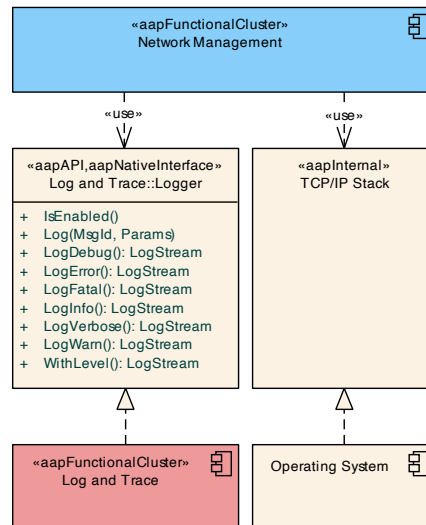


Figure 9.30: Interfaces required by Network Management

Interface	Purpose
TCP/IP Stack	Network Management should use this interface to set and to determine the status of IP-based networks.
Log and Trace::Logger	Network Management shall use this interface to log standardized messages.
Registry::ManifestAccessor	Network Management shall use this interface to read information about NmNetworkHandles from the Manifests.

Table 9.11: Interfaces required by Network Management

9.3.3 Time Synchronization

Name:	Time Synchronization
Short Name:	tsync
Category:	Communication
Daemon-based:	Yes
Responsibilities:	Time Synchronization provides synchronized time information in distributed applications. Synchronized time information between different applications and/or Machines is of paramount importance when the correlation of different events across a distributed system is needed, either to be able to track such events in time or to trigger them at an accurate point in time.

9.3.3.1 Defined interfaces

The interfaces of [Time Synchronization](#) are categorized into interfaces for providing time information (see Section [9.3.3.1.1](#)) and interfaces for consuming time information (see Section [9.3.3.1.2](#)).

9.3.3.1.1 Interfaces for time base providers

[Time Synchronization](#) defines the [SynchronizedTimeBaseProvider](#) and [OffsetTimeBaseProvider](#) interfaces to provide time information for a synchronized time base.

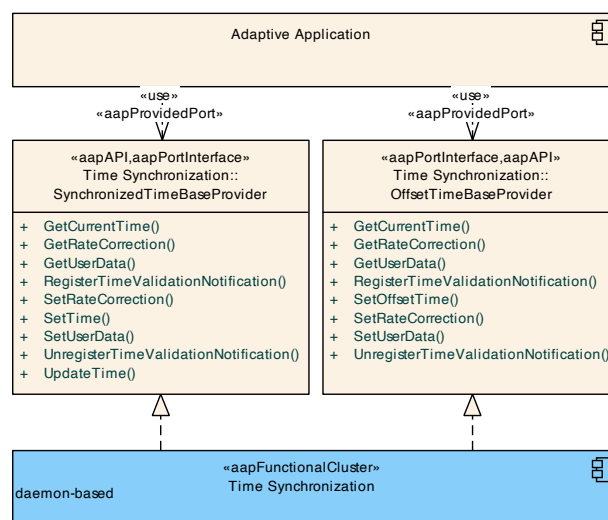


Figure 9.31: Interfaces for time base providers

Name:	SynchronizedTimeBaseProvider	
Technology:	Port interface	
Generated:	No	
Meta-model interface type:	SynchronizedTimeBaseProviderInterface	
Usage:	Public API	
Description:	Provides access to a synchronized time base. It allows to get the current time point, the rate deviation, the current status and the received user data.	
Operations:	GetCurrentTime	Obtain the current time (regardless of the current sync status).
	GetRateCorrection	Obtain the current rate deviation of the clock.
	GetUserData	Get the user defined data of the time base.
	RegisterTimeValidationNotification	Used by time provider applications to receive time sync parameters. A maximum of one notifier can be registered. Every further registration overwrites the current registration.
	SetRateCorrection	Set the rate correction that will be applied to time values.
	SetTime	Set a new time value for the clock. Setting a new time also triggers transmission on the bus.





	SetUserData	Set the user data of the time base.
	UnregisterTimeValidationNotification	Used by time provider applications to receive time sync parameters.
	UpdateTime	Set a new time value for the clock. The clock value is only updated locally, transmission on the bus will happen in the next cycle.

Name:	OffsetTimeBaseProvider	
Technology:	Port interface	
Generated:	No	
Meta-model interface type:	SynchronizedTimeBaseProviderInterface	
Usage:	Public API	
Description:	Provides access to the offset time base. It allows to get the current time point, the rate deviation, the current status and the received user data.	
Operations:	GetCurrentTime	Get the current time (regardless of the current sync status).
	GetRateCorrection	Obtain the current rate deviation of the clock.
	GetUserData	Get the user defined data of the time base.
	RegisterTimeValidationNotification	Used by time provider applications to receive time sync parameters. A maximum of one notifier can be registered. Every further registration overwrites the current registration.
	SetOffsetTime	Set a new offset time value for the clock. Setting a new time also triggers transmission on the bus.
	SetRateCorrection	Set the rate correction that will be applied to time values.
	SetUserData	Set the user data of the time base.
	UnregisterTimeValidationNotification	Un-register the notifier to receive time sync parameters.

9.3.3.1.2 Interfaces for time base consumers

[Time Synchronization](#) defines the [SynchronizedTimeBaseConsumer](#) interface to retrieve time information for a synchronized time base. [SynchronizedTimeBaseStatus](#) is used to determine the status of a synchronized time base.

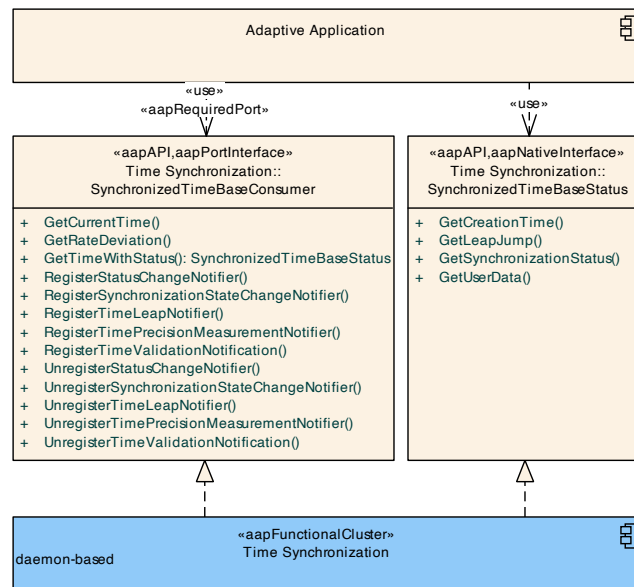


Figure 9.32: Interfaces for time base consumers

Name:	SynchronizedTimeBaseConsumer	
Technology:	Port interface	
Generated:	No	
Meta-model interface type:	SynchronizedTimeBaseConsumerInterface	
Usage:	Public API	
Description:	Provides access to the synchronized time base. It allows to get the current time point, the rate deviation, the current status and the received user data.	
Operations:	GetCurrentTime	Obtain the current time (regardless of the current sync status).
	GetRateDeviation	Obtain the current rate deviation of the clock.
	GetTimeWithStatus	Obtain a snapshot of the current state of the clock. This includes status flags, clock configuration and the actual time value of the created status object.
	RegisterStatusChangeNotifier	Register a notifier function which is called if a status flag is changed (i.e. synchronization state, time leap or userdata). A maximum of one notifier can be registered. Every further registration overwrites the current registration.
	RegisterSynchronizationStateChangeNotifier	Register a notifier function which is called if a synchronization state is changed. A maximum of one notifier can be registered. Every further registration overwrites the current registration.
	RegisterTimeLeapNotifier	Register a notifier function which is called if a time leap happened. A maximum of one notifier can be registered. Every further registration overwrites the current registration.
	RegisterTimePrecisionMeasurementNotifier	Register a notifier function which is called if a new time precision snapshot is available. A maximum of one notifier can be registered. Every further registration overwrites the current registration. Time Synchronization will not do any queuing. If needed it has to be done within the notifier.





	RegisterTimeValidationNotification	Used by time consumer applications to receive time sync parameters. A maximum of one notifier can be registered. Every further registration overwrites the current registration.
	UnregisterStatusChangeNotifier	Un-register a notifier function which is called if a status flag is changed (i.e. synchronization state, time leap or userdata).
	UnregisterSynchronizationStateChangeNotifier	Un-register a notifier function which is called if a synchronization state is changed.
	UnregisterTimeLeapNotifier	Un-register a notifier function which is called if a time leap happened.
	UnregisterTimePrecisionMeasurementNotifier	Un-register a notifier function which is called if a new time precision snapshot is available.
	UnregisterTimeValidationNotification	Un-register a notifier function for receiving time sync parameters.

Name:	SynchronizedTimeBaseStatus	
Technology:	Native interface	
Usage:	Public API	
Description:	Represents a snapshot of a time point including its states.	
Operations:	GetCreationTime	Obtain the creation time of this object.
	GetLeapJump	Determine the direction of a leap jump. Only the jump until the previous object creation is included.
	GetSynchronizationStatus	Returns the synchronization state when the object was created.
	GetUserData	Returns the user defined data of the time base.

9.3.3.2 Provided interfaces

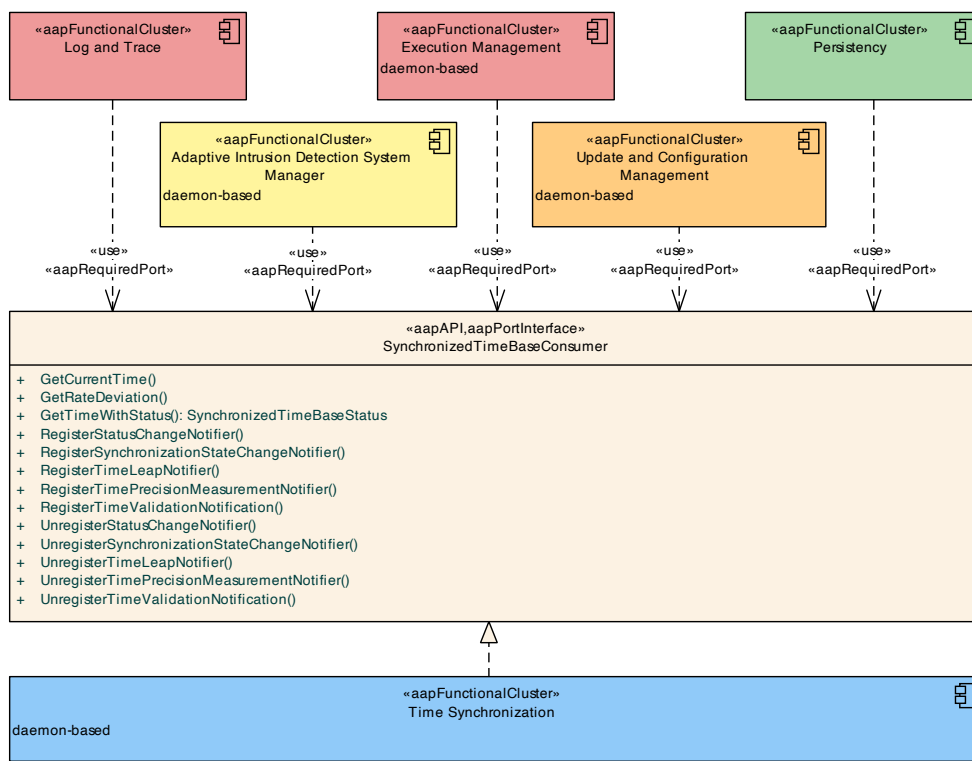


Figure 9.33: Users of Time Synchronization interfaces

Interface	Requiring functional clusters
Time Synchronization::SynchronizedTimeBase Consumer	Adaptive Intrusion Detection System Manager
	Execution Management
	Log and Trace
	Persistency
	Update and Configuration Management

Table 9.12: Interfaces provided by Time Synchronization to other Functional Clusters

9.3.3.3 Required interfaces

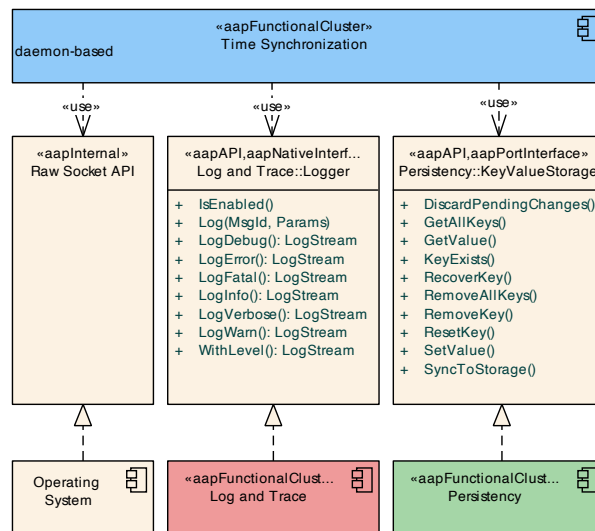


Figure 9.34: Interfaces required by Time Synchronization

Interface	Purpose
Raw Socket API	Time Synchronization should use this interface to send and receive raw ethernet packets as required by the time synchronization protocol.
Execution Management::ExecutionClient	Time Synchronization shall use this interface to report the state of its daemon process.
Log and Trace::Logger	Time Synchronization shall use this interface to log standardized messages.
Persistency::KeyValueStorageOperations	Time Synchronization should use this interface to persist the last received timestamp to enable a faster startup.
Persistency::KeyValueStorage	Time Synchronization should use this interface to persist the last received timestamp to enable a faster startup.
Platform Health Management::SupervisedEntity	Time Synchronization should use this interface to enable supervision of its daemon process by Platform Health Management
Registry::ManifestAccessor	Time Synchronization shall use this interface to read information about TimeBaseResources as well as their providers and consumers from the Manifests.

Table 9.13: Interfaces required by Time Synchronization

9.4 Storage

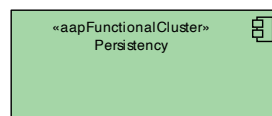


Figure 9.35: Functional Clusters in category Storage

Interface	Purpose
Execution Management::ExecutionClient	This interface shall be used by the daemon process(es) inside Update and Configuration Management to report their execution state to Execution Management .
Log and Trace::Logger	Update and Configuration Management shall use this interface to log standardized messages.
Persistency::FileStorage	This interface should be used to store files, for example downloaded packages.
Persistency::KeyValueStorage	This interface should be used to store internal state of Update and Configuration Management .
Persistency::ReadAccessor	This interface should be used to store files, for example downloaded packages.
Persistency::ReadWriteAccessor	This interface should be used to store files, for example downloaded packages.
Platform Health Management::SupervisedEntity	This interface should be used to supervise the daemon process(es) of Update and Configuration Management .
Registry::ManifestAccessor	Update and Configuration Management shall use this interface to read information about its configuration from the ManifestS .
Time Synchronization::SynchronizedTimeBase Consumer	Update and Configuration Management shall use this interface to get latest timestamp.

Table 9.25: Interfaces required by Update and Configuration Management

9.7.2 Registry

Name:	Registry
Short Name:	n/a
Category:	Configuration
Daemon-based:	No
Responsibilities:	The Registry is an internal component of the AUTOSAR Adaptive Platform that provides access the information stored in Manifests . It is not intended to be used by Adaptive Applications directly.

9.7.2.1 Defined interfaces

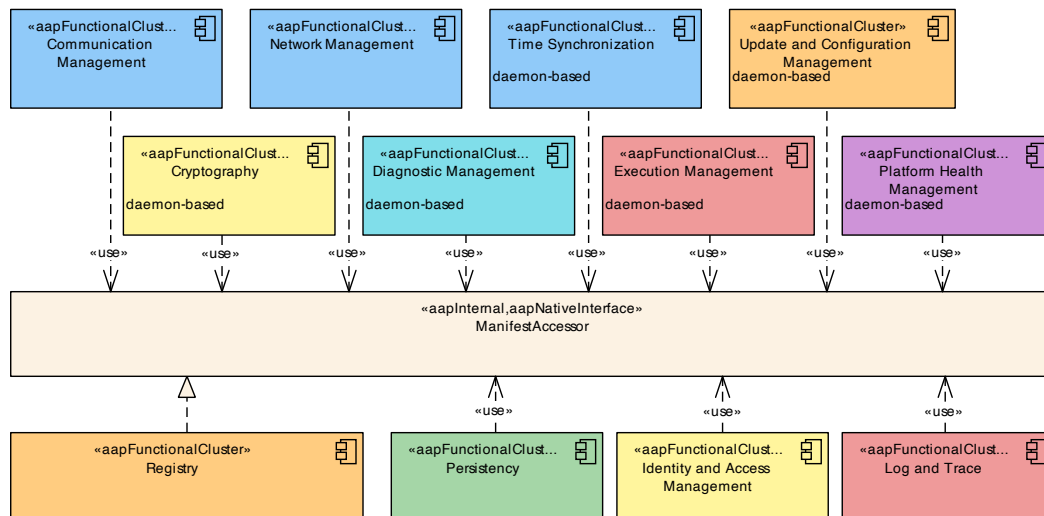


Figure 9.81: Interfaces of Registry

Name:	ManifestAccessor
Technology:	Native interface
Usage:	Internal
Description:	This interface provides functionality to read information that was modeled in the <code>Manifest(s)</code> . This interface is not detailed in this document.

9.7.2.2 Provided interfaces

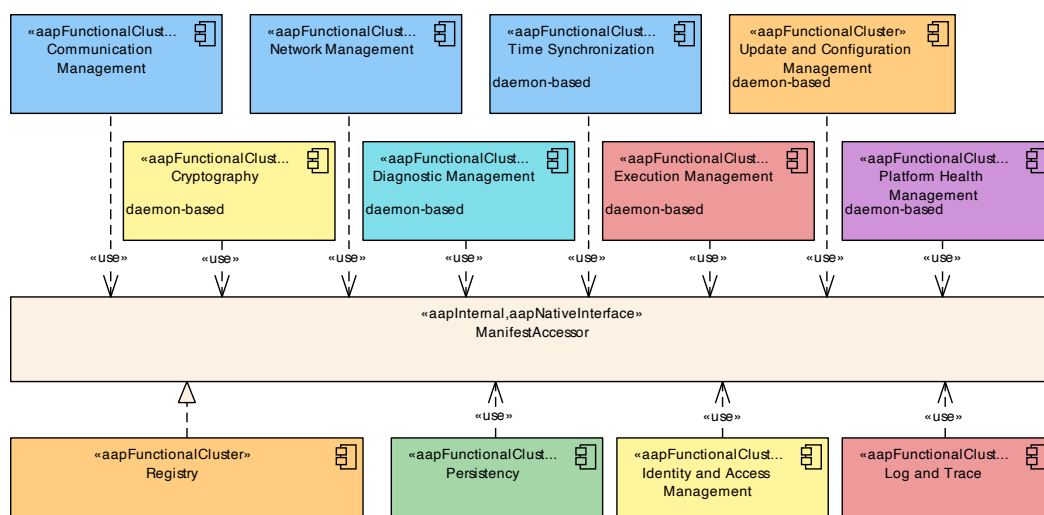


Figure 9.82: Users of the Registry interfaces

Interface	Requiring functional clusters
Registry::ManifestAccessor	Communication Management
	Cryptography
	Diagnostic Management
	Execution Management
	Identity and Access Management
	Log and Trace
	Network Management
	Persistency
	Platform Health Management
	Time Synchronization
	Update and Configuration Management

Table 9.26: Interfaces provided by Registry to other Functional Clusters

9.7.2.3 Required interfaces

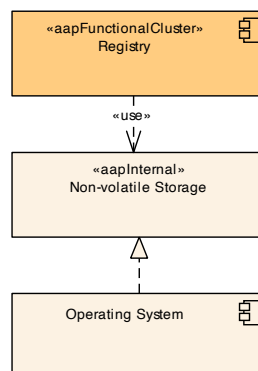


Figure 9.83: Interfaces required by Registry

Interface	Purpose
Non-volatile Storage	Registry shall use this interface to read the information from the Manifest(s).

Table 9.27: Interfaces required by Registry

9.8 Diagnostics

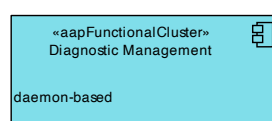


Figure 9.84: Functional Clusters in category Diagnostics



	RequestDownloadExit	Called for RequestTransferExit.
	StopOffer	Disable forwarding of request messages from Diagnostic Management.

9.8.1.1.4 Interfaces for State Management

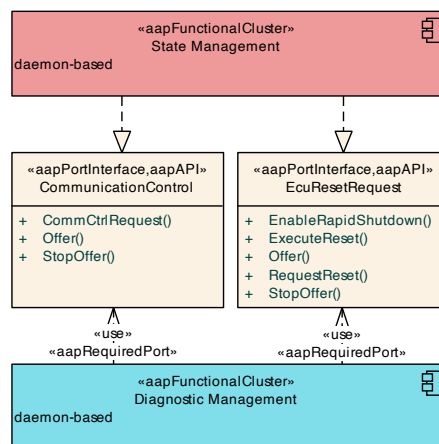


Figure 9.92: Interfaces for State Management

Name:	CommunicationControl	
Technology:	Port interface	
Generated:	No	
Meta-model interface type:	DiagnosticComControl	
Usage:	Public API	
Description:	This interface provides functionality for CommunicationControl.	
Operations:	CommCtrlRequest	Called for CommunicationControl (0x28) with any subfunction as subfunction value is part of argument list.
	Offer	Enable forwarding of request messages from Diagnostic Management.
	StopOffer	Disable forwarding of request messages from Diagnostic Management.

Name:	EcuResetRequest	
Technology:	Port interface	
Generated:	Yes	
Meta-model interface type:	DiagnosticEcuResetInterface	
Usage:	Public API	
Description:	This interface provides functionality for EcuReset requests.	
Operations:	EnableRapidShutdown	Called for subfunction En-/DisableRapidShutdown.
	ExecuteReset	Execute the requested reset.



12 Deployment View

This chapter provides an overview of exemplary deployment scenarios for an AUTOSAR Adaptive Platform. Since the AUTOSAR Adaptive Platform is highly configurable in its deployment, this section rather provides constraints on supported deployments and a selection of relevant deployment scenarios.

12.1 Vehicle Software Deployment

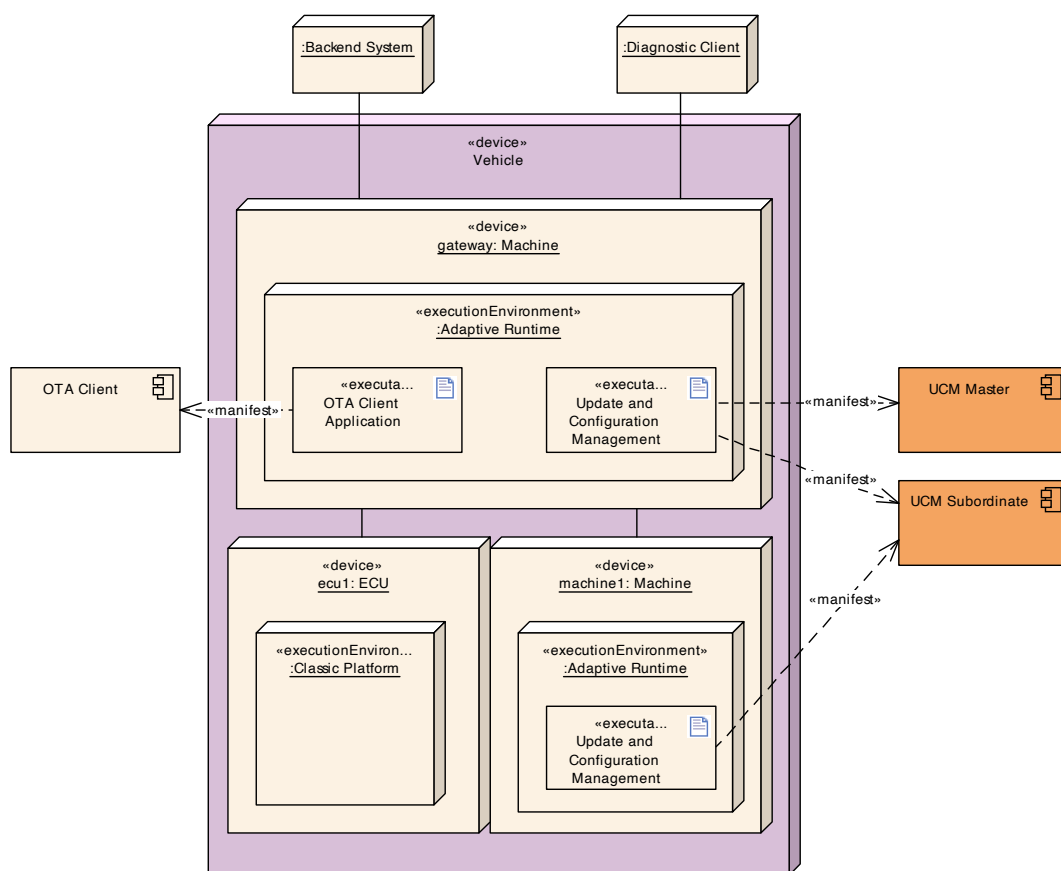


Figure 12.1: Exemplary vehicle software update scenario

Update and Configuration Management allows to install and update software on the AUTOSAR Adaptive Platform and AUTOSAR Classic Platform. For the AUTOSAR Adaptive Platform, Update and Configuration Management also allows to remove software. The software packages can be received either from a Diagnostic Client or from a specific Backend System for over-the-air updates. In a vehicle, one Adaptive Application takes the role of a master that controls the update process in the vehicle and distributes individual software packages to the Machines and ECUs within a vehicle.

12.2 Deployment of Software Packages on a Machine

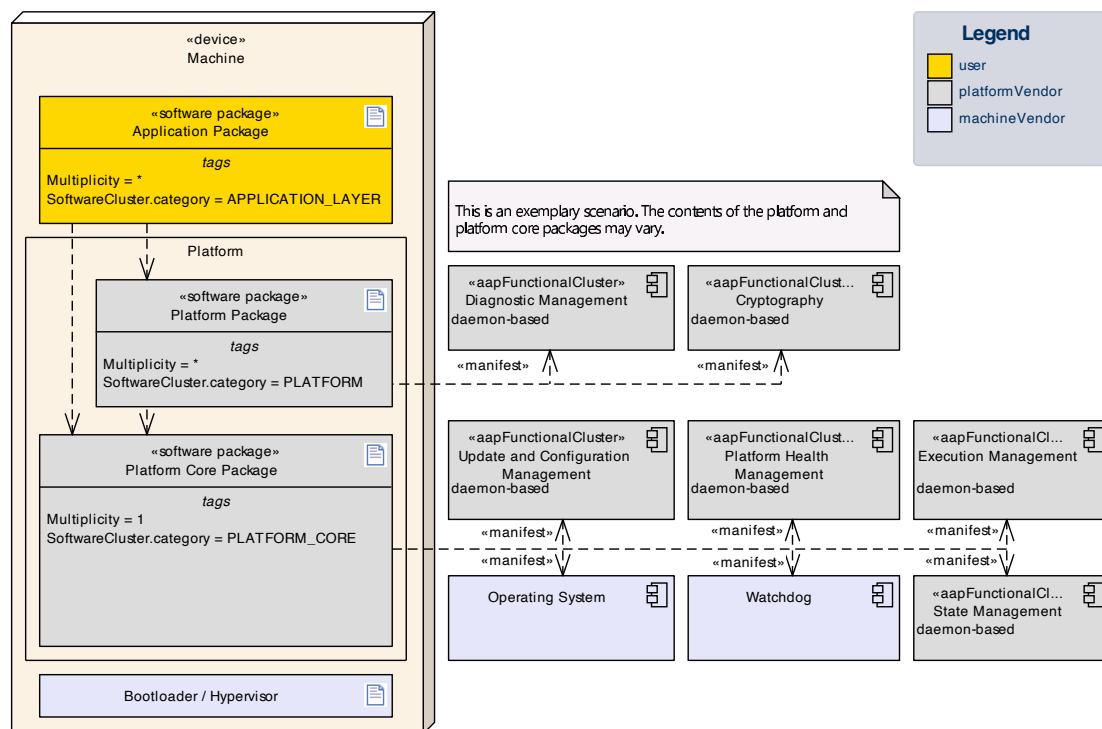


Figure 12.2: Exemplary deployment of Software Packages on a Machine

[Update and Configuration Management](#) manages the software installed on a Machine using one or more Software Packages. The configuration of the contents of a Software Package is done in the Manifest by means of a Software Cluster. Software Cluster defines the enumeration attribute category with the values APPLICATION, PLATFORM, and PLATFORM_CORE. Some rules apply on the use of the Software Cluster category and corresponding Software Packages which are outlined in the following.

For a Machine there shall exist exactly one Software Cluster with category PLATFORM_CORE. The corresponding Platform Core Package needs to include all required core components provided by the Platform Vendor and Machine Vendor, for example the operating system, device drivers, and the required Functional Clusters of the Adaptive Platform (see Figure 12.2). It is assumed that at least the Functional Clusters [Execution Management](#), [State Management](#), and [Update and Configuration Management](#) are part of the Platform Core Package since they are required to install any additional Software Packages. However, the concrete extend of a Platform Core Package is vendor and/or project specific. A Platform Core Package cannot be removed. The Software Cluster with category PLATFORM_CORE needs to be self-contained and therefore shall not have dependencies to other Software Clusters.

Optional Functional Clusters of the Adaptive Platform may be distributed as part of any number of additional Software Clusters with category PLATFORM within Platform Packages. Such Software Clusters may depend on the the

Software Cluster with category PLATFORM_CORE (see Figure 12.2). Platform Packages may be installed and removed as needed.

Application components should be distributed as part of additional Software Clusters with category APPLICATION within Application Packages. Such Software Clusters may depend on the on Software Clusters with any kind of category (see Figure 12.2). Application Packages may be installed and removed as needed.

The Platform Core Package and Platform Packages shall include all required Functional Cluster daemons and their respective configuration (e.g., startup configuration). This frees applications (distributed as Application Packages) from taking care of the configuration of the platform.

13 Cross-cutting Concepts

This section provides an overview of cross-cutting concepts and patterns used in the AUTOSAR Adaptive Platform.

13.1 Overview of Platform Entities

The AUTOSAR Adaptive Platform defines design entities that several `Functional Clusters` depend on. Figure 13.1 provides an overview of these entities, their logical relationships, and the `Functional Clusters` that depend on them. For the sake of brevity, this overview uses simplifications and abstractions over the actual specifications in the [9, manifest specification].

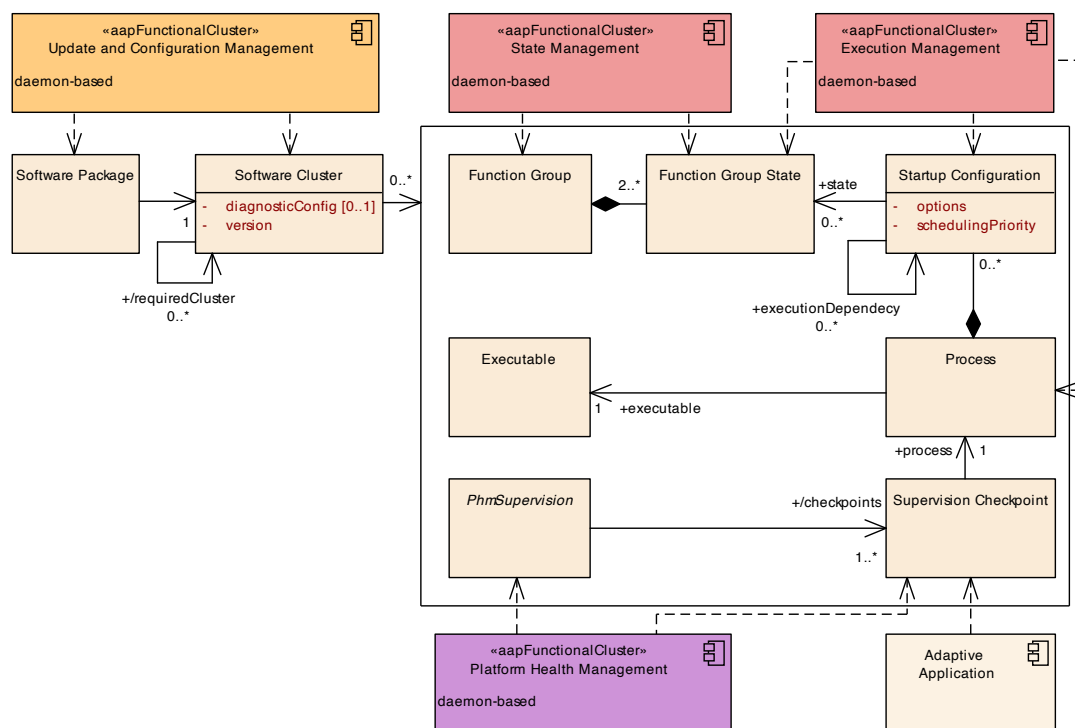


Figure 13.1: Overview of platform entities and their logical relationships

A `Software Package` is a digitally signed package that can be installed/uninstalled via `Update and Configuration Management`. A `Software Package` contains exactly one `Software Cluster` (see Section 13.4 for details). A `Software Cluster` refers to a set of `Executables` (and other files). The corresponding executable file then holds the executable code for the `Machine` that the AUTOSAR Adaptive Platform runs on.

Additionally, a `Software Cluster` configuration collects a set of `Processes` (cf. Section 13.4) and related entities. A `Process` refers to an `Executable` and provides different `Startup Configuration` values, for example parameters, a scheduling

priority, and resource constraints. A Startup Configuration of a Process applies to one or more Function Group States. Function Group States belong to a Function Group.

During runtime, State Management requests to enter a Function Group State from Execution Management. Execution Management then terminates and starts the Processes accordingly using the underlying Operating System.

For safety-critical systems, Platform Health Management performs supervision of Processes according to rules (logical sequence, deadlines) defined in PhmSupervisions. A PhmSupervision refers to a number of Supervision Checkpoints. During runtime, a process reports whenever it has reached such a checkpoint in its control flow.

13.2 Function Group

A Function Group is (logically) made up a set of modeled Processes that provide a certain functionality. For example, a Function Group could be an application, or a service. A special Function Group is the Machine State that groups the Process of the AUTOSAR Adaptive Platform itself. A Function Group contains a set of Function Group States.

13.3 Function Group State

A Function Group State defines which Processes of a Function Group with what configuration parameters shall be running or not. The Machine State (that refers to the Processes of the AUTOSAR Adaptive Platform itself) defines at least the following Function Group States: Off, Startup, Shutdown, and Restart.

13.4 Software Cluster

A Software Cluster configuration refers to a set of modeled Processes. Those Processes are (transitively) used by one or more Function Group(s). Hereby, a Function Group and its associated entities shall be part of only one Software Cluster. In other words, Function Groups that span several Software Clusters are invalid. A Software Cluster is packaged into one Software Package - the atomic installable/updateable unit managed by Update and Configuration Management. A Software Cluster may depend on other Software Clusters. Such dependencies are expressed by version constraints. A Software Cluster may also specify structural dependencies to Sub Software Clusters in order to build larger installable units. The top of such a structural dependency hierarchy is called a Root Software Cluster. Please note that a Software Cluster is only used to configure deployment aspects. A Software Cluster is not a runtime entity.

A Root Software Cluster may specify a diagnostic configuration, in particular, a diagnostic address. In contrast, a Sub Software Cluster may depend on a diagnostic configuration of its Root Software Cluster. The diagnostic configuration applies to Processes that are (transitively) contained in a Root Software Cluster and its Sub Software Cluster(s). That means, at runtime, any diagnostic event produced by those Processes will be associated with the diagnostic address. Please refer to Section 13.11 for further details on the diagnostic deployment.

An exemplary Software Cluster during application design is shown in Figure 13.2. The application Software Cluster(s) are modeled/configured in the same way as the platform Software Cluster by defining Function Groups with Function Group States and associating StartupConfigurations of Processes to them.

A Software Cluster serves as a grouping entity during application design. As a result, entities within a Software Cluster, in particular the Function Groups, do not need to have a unique (simple) name within the overall model because their path is still unique. This allows to design Software Clusters independently, for example, by external suppliers.

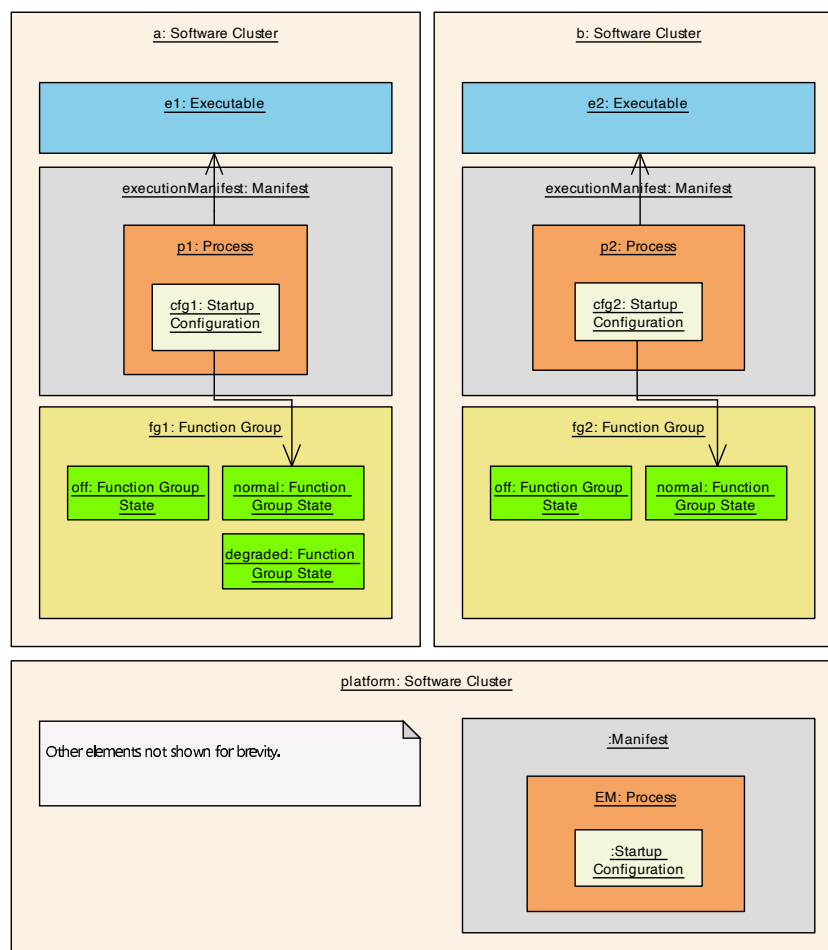


Figure 13.2: Exemplary Software Cluster during application design

From such a standardized model, an equivalent implementation-specific configuration for **Execution Management** is derived that is used during runtime (see Figure 13.3). That configuration advises **Execution Management** to start and configure processes accordingly when a **Function Group State** is requested. Hereby, **Execution Management** (logically) merges configurations contributed by all installed **Software Packages**. Other **Functional Clusters** that depend on configuration provided in the **Manifests** merge the configurations contributed by all installed **Software Packages** in the same way. Please also note that there is no corresponding runtime entity for a **Software Cluster** (see Figure 13.3).

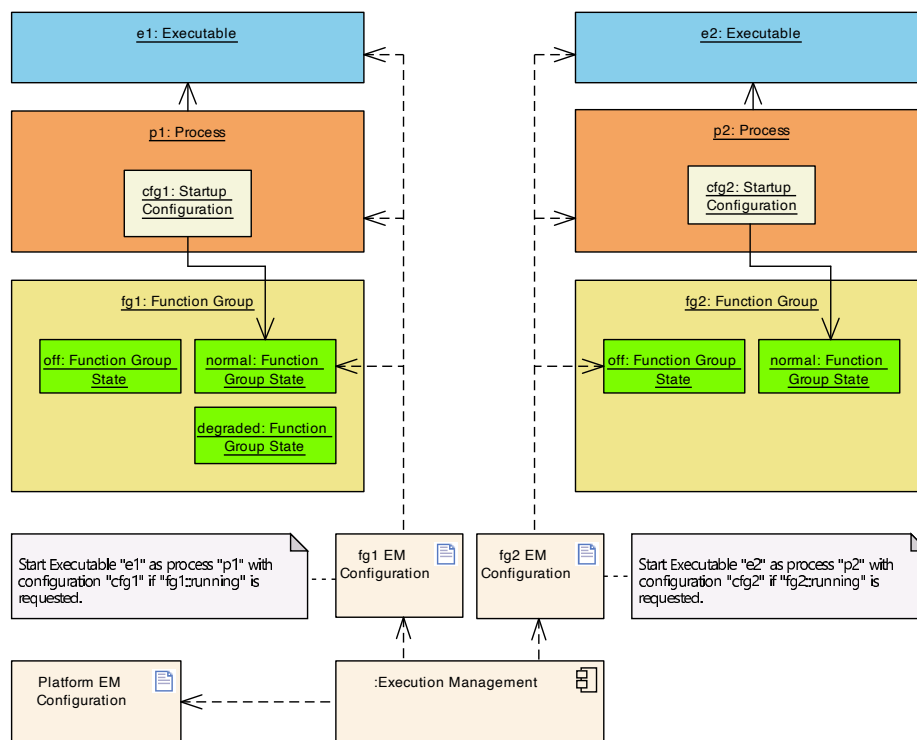


Figure 13.3: Impact of exemplary Software Cluster during runtime

All Processes related to the Functional Clusters of the AUTOSAR Adaptive Platform should be referenced only in Software Clusters of category PLATFORM_CORE or PLATFORM. This allows for platform-independent development of Software Clusters of category APPLICATION_LAYER.

In case a Functional Cluster may need multiple logical instances (for example, Diagnostic Management has a logical instance per diagnostic address), an implementation of the Functional Cluster should still use a single physical (daemon) process.

An AUTOSAR Adaptive Platform vendor may deviate from this design guide but should provide additional countermeasures to keep **Adaptive Applications** portable.

13.5 Machine

The AUTOSAR Adaptive Platform regards hardware it runs on as a `Machine`. The rationale behind that is to achieve a consistent platform view regardless of any virtualization technology which might be used. The `Machine` might be a real physical machine, a fully-virtualized machine, a para-virtualized OS, an OS-level-virtualized container or any other virtualized environment.

On hardware, there can be one or more `Machine`, and only a single instance of AUTOSAR Adaptive Platform runs on a machine. It is generally assumed that this hardware includes a single chip, hosting a single or multiple `Machines`. However, it is also possible that multiple chips form a single `Machine` if the AUTOSAR Adaptive Platform implementation allows it.

13.6 Manifest

A `Manifest` represents a piece of AUTOSAR model description that is created to support the configuration of an AUTOSAR Adaptive Platform product and which is uploaded to the AUTOSAR Adaptive Platform product, potentially in combination with other artifacts (like binary files) that contain executable code to which the `Manifest` applies. Please note that a typical `Adaptive Application` will make use of several distinct but interrelated `Manifests`. Hereby, the individual `Manifests` contribute information to the complete application model. For example, each `Software Cluster` may contribute a self-contained set of `Manifests` to configure its functionality.

The usage of a `Manifest` is limited to the AUTOSAR Adaptive Platform. This does not mean, however, that all ARXML produced in a development project that targets the AUTOSAR Adaptive Platform is automatically considered a `Manifest`. In fact, the AUTOSAR Adaptive Platform is usually not exclusively used in a vehicle project. A typical vehicle will most likely be also equipped with a number of ECUs developed on the AUTOSAR Classic Platform and the system design for the entire vehicle will, therefore, have to cover both, ECUs built on top of the AUTOSAR Classic Platform and `Machines` created on top of the AUTOSAR Adaptive Platform.

In principle, the term `Manifest` could be defined such that there is conceptually just one "Manifest" and every deployment aspect would be handled in this context. This does not seem appropriate because it became apparent that `Manifest`-related model-elements exist that are relevant in entirely different phases of a typical development project.

This aspect is taken as the main motivation that next to the application design it is necessary to subdivide the definition of the term `Manifest` in three different partitions:

Application Design This kind of description specifies all design-related aspects that apply to the creation of application software for the AUTOSAR Adaptive Platform. It is not necessarily required to be deployed to the adaptive platform machine, but the application design aids the definition of the deployment of application software in the

Execution Manifest and Service Instance Manifest. See Section [13.7](#) for details.

Execution Manifest This kind of `Manifest` is used to specify the deployment-related information of applications running on the AUTOSAR Adaptive Platform. An `Execution Manifest` is bundled with the actual executable code to support the integration of the executable code onto the machine. See Section [13.8](#) for details.

Service Instance Manifest This kind of `Manifest` is used to specify how service-oriented communication is configured in terms of the requirements of the underlying transport protocols. A `Service Instance Manifest` is bundled with the actual executable code that implements the respective usage of service-oriented communication. See Section [13.9](#) for details.

Machine Manifest This kind of `Manifest` is supposed to describe deployment-related content that applies to the configuration of just the underlying machine (i.e. without any applications running on the machine) that runs an AUTOSAR Adaptive Platform. A `Machine Manifest` is bundled with the software taken to establish an instance of the AUTOSAR Adaptive Platform. See Section [13.10](#) for details.

The temporal division between the definition (and usage) of different kinds of `Manifest` leads to the conclusion that in most cases different physical files will be used to store the content of the three kinds of `Manifest`. In addition to the Application Design and the different kinds of `Manifest`, the AUTOSAR Methodology supports a System Design with the possibility to describe Software Components of both AUTOSAR Platforms that will be used in a System in one single model. The Software Components of the different AUTOSAR platforms may communicate in a service-oriented way with each other. But it is also possible to describe a mapping of Signals to Services to create a bridge between the service-oriented communication and the signal-based communication.

13.7 Application Design

The application design describes all design-related modeling that applies to the creation of application software for the AUTOSAR AP. Application Design focuses on the following aspects:

- Data types used to classify information for the software design and implementation
- Service interfaces as the pivotal element for service-oriented communication
- Definition how service-oriented communication is accessible by the application
- Persistency Interfaces as the pivotal element to access persistent data and files
- Definition how persistent storage is accessible by the application
- Definition how files are accessible by the application

- Definition how crypto software is accessible by the application
- Definition how the Platform Health Management is accessible by the application
- Definition how Time Bases are accessible by the application
- Serialization properties to define the characteristics of how data is serialized for the transport on the network
- Description of client and server capabilities
- Grouping of applications in order to ease the deployment of software.

The artifacts defined in the application design are independent of a specific deployment of the application software and thus ease the reuse of application implementations for different deployment scenarios.

13.8 Execution Manifest

The purpose of the execution manifest is to provide information that is needed for the actual deployment of an application onto the AUTOSAR AP. The general idea is to keep the application software code as independent as possible from the deployment scenario to increase the odds that the application software can be reused in different deployment scenarios. With the execution manifest the instantiation of applications is controlled, thus it is possible to

- instantiate the same application software several times on the same machine, or to
- deploy the application software to several machines and instantiate the application software per machine.

The Execution manifest focuses on the following aspects:

- Startup configuration to define how the application instance shall be started. The startup includes the definition of startup options and access roles. Each startup may be dependent on machines states and/or function group states.
- Resource Management, in particular resource group assignments.

13.9 Service Instance Manifest

The implementation of service-oriented communication on the network requires configuration which is specific to the used communication technology (e.g. SOME/IP). Since the communication infrastructure shall behave the same on the provider and the requesters of a service, the implementation of the service shall be compatible on both sides.

The Service Instance Manifest focuses on the following aspects:

- Service interface deployment to define how a service shall be represented on the specific communication technology.
- Service instance deployment to define for specific provided and required service instances the required credentials for the communication technology.
- The configuration of E2E protection
- The configuration of Security protection
- The configuration of Log and Trace

13.10 Machine Manifest

The machine manifest allows to configure the actual adaptive platform instance running on specific hardware (machine).

The Machine Manifest focuses on the following aspects:

- Configuration of the network connection and defining the basic credentials for the network technology (e.g. for Ethernet this involves setting of a static IP address or the definition of DHCP).
- Configuration of the service discovery technology (e.g. for SOME/IP this involves the definition of the IP port and IP multi-cast address to be used).
- Definition of the used machine states.
- Definition of the used function groups.
- Configuration of the adaptive platform functional cluster implementations (e.g. the operating system provides a list of OS users with specific rights).
- The configuration of the Crypto platform Module.
- The configuration of Platform Health Management.
- The configuration of Time Synchronization.
- Documentation of available hardware resources (e.g. how much RAM is available; how many processor cores are available).

13.11 Diagnostics deployment

A diagnostic configuration, in particular a diagnostic address, may only be assigned to a `Root Software Cluster` in the `Manifest`. Nevertheless, the mapped `DiagnosticContributionSet(s)` may be distributed across several `Software Clusters`. This concept provides a lot of flexibility in assignment of a single diagnostic address to `Software Clusters`. For example, in one extreme this allows to use a single diagnostic address for the whole `Machine` (see Figure 13.4), in another extreme

a single diagnostic address per Software Cluster could be used (see Figure 13.5). Deployment scenarios in between those extremes are also possible.

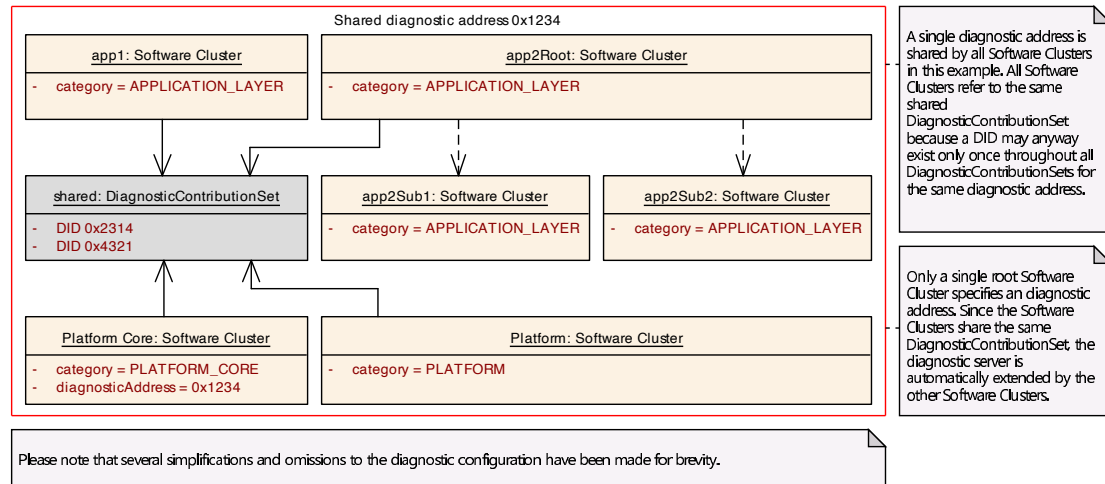


Figure 13.4: Example defining a single diagnostic address for the whole Machine

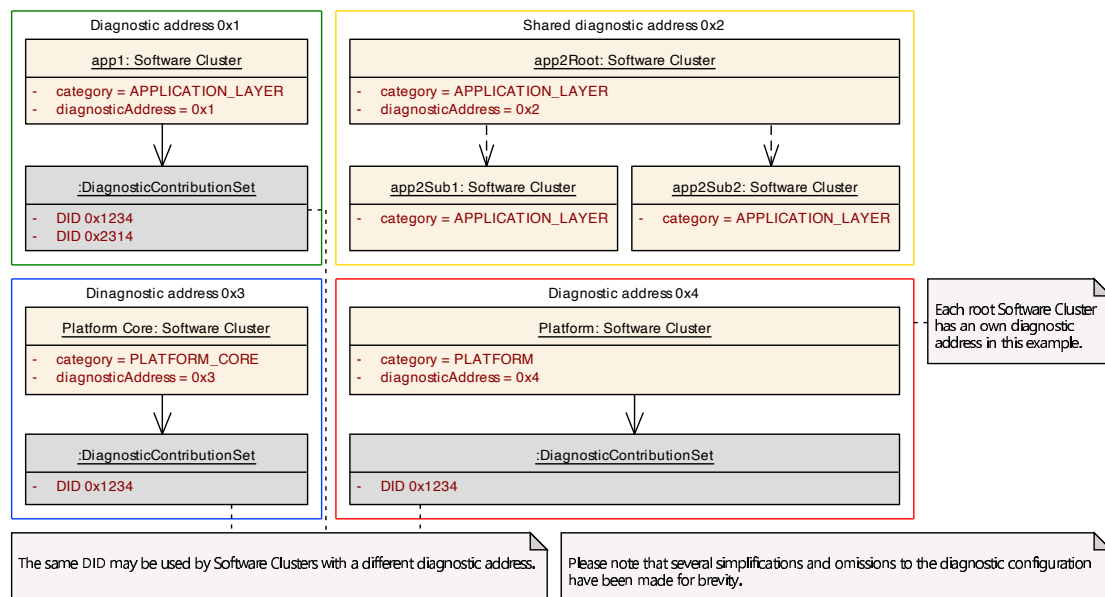


Figure 13.5: Example using one diagnostic address for each Software Cluster

In the case of a distributed `DiagnosticContributionSet`, each Software Cluster shall include its related diagnostic configuration objects (for example, the Data Identifier configuration). The merge of such a split `DiagnosticContributionSet` is done internally by the AUTOSAR Adaptive Platform (e.g. during installation or during start up the Diagnostic Management daemon).

13.12 Error Handling

Proper handling of errors during runtime is an important aspect to build safe and secure systems. The AUTOSAR Adaptive Platform does provide means for raising and handling of such errors on different levels in the platform.

`Platform Health Management` detects errors (errors in the logical control flow, missed deadlines, and missed liveness reporting) at the level of `Processes` and performs recovery actions (for example, degradation) according to rules defined in the Manifest. `Execution Management` detects unexpected termination of `Processes` and reports to `State Management` for handling of such errors.

During execution of a `Process` of an `Adaptive Application`, different abnormal conditions might be detected and need to be handled and/or reported. The following types of unsuccessful operations are distinguished within the AUTOSAR Adaptive Platform:

- An `Error` is the inability of an AUTOSAR Runtime for Adaptive Applications API function to fulfill its specified purpose. An `Error` it is often a consequence of invalid and/or unexpected input data. An `Error` is considered to be recoverable and therefore shall be handled by applications.
- A `Violation` is the consequence of failed pre- or post-conditions of internal state of the AUTOSAR Runtime for Adaptive Applications. A `Violation` is considered to be non-recoverable.
- A `Corruption` is the consequence of the corruption of a system resource, e.g. stack or heap overflow, or a hardware memory flaw (for example, a detected bit flip). A `Corruption` is considered to be non-recoverable.
- A failed default allocation is the inability of the AUTOSAR Runtime for Adaptive Applications's default memory allocation mechanism to satisfy an allocation request (for example, there is not enough free memory available).

It is expected that a `Violation` or `Corruption` will not be experienced by a user of the AUTOSAR Adaptive Platform (i.e. an application developer), unless there is something seriously wrong in the overall system. For example, faulty hardware may lead to a `Corruption`. A `Violation` may occur if basic assumptions about resource requirements are violated, or the user runs the AUTOSAR Runtime for Adaptive Applications in a configuration that is not supported by its vendor.

13.13 Trusted Platform

To guarantee the correct function of the system, it is crucial to ensure that the code executed on the AUTOSAR Adaptive Platform is unaltered (integrity) and has legitimate origin (authenticity). Keeping this property allows the integrator to build a `Trusted Platform`. A key property of a system that implements a `Trusted Platform` is a `Trust Anchor` (also called `Root of Trust`). A `Trust Anchor` is often realized

as a public key that is stored in a secure environment, e.g. in non-modifiable persistent memory or in an `Hardware Security Module`. A system designer is responsible to ensure that the system starts beginning with a `Trust Anchor` and that the chain of trust is kept until the `Execution Management` is launched. Depending on the mechanism that is chosen by the system designer to establish the chain of trust, the integrity and authenticity of the entire system (including all executables) may be checked during system start-up. Alternatively, the system designer might only ensure that the already executed software has been checked regarding integrity and authenticity and `Execution Management` takes over responsibility on continuing the chain of trust when it takes over control of the system. In the latter case, the system integrator is responsible to ensure that the `Execution Management` is configured accordingly.

Passing trust requires that a trusted entity checks (using trusted functionality) that the entity down the chain is authentic. The `Trust Anchor` (the first entity in the chain) is authentic by definition. An example of such a chain of trust could look like this: The `Trust Anchor` authenticates the bootloader before the bootloader is being started. In each subsequent step in the boot process, the to-be-started executable shall be authenticated first, for example by the executable started previously or by some external entity like an `Hardware Security Module`. After the relevant parts of the `Operating System` have been authentically started, it shall launch `Execution Management` as one of its first processes in the same manner passing trust to the AUTOSAR Adaptive Platform. Then, `Execution Management` takes over the responsibility of authenticating `Adaptive Applications` before launching them.

As stated above, if authenticity is not checked by the functionality of the `Trust Anchor` itself, which is authentic by definition, the functionality that shall be applied to verify authenticity of an executable has to be authenticated as well before it is applied. For instance, if the `Crypto Functional Cluster` shall be used to verify authenticity of executables, the `Crypto Functional Cluster` itself shall be authenticated by some trusted entity before it is used.

13.14 Secure Communication

AUTOSAR supports different protocols that provide communication security over a network. Integrity of messages can be ensured by the end-to-end protection offered by the [10, AUTOSAR E2E library]. End-to-end protection assumes that safety- and security-related data exchange shall be protected at runtime against the effects of faults within the communication link. Such faults include random hardware faults (e.g. corrupt registers of a transceiver), interference (e.g. electromagnetic interference), and systematic faults in the communication stack. The configuration of end-to-end-protection is done via `Service Instance Manifest` on level of `Service` events, methods, and fields (notifier, get, and set methods). Confidentiality and authenticity of messages can be ensured by dedicated configurations for the individual transport protocols (e.g. TLS, SecOC) in the `Service Instance Manifest` on level of `Service` events, methods, and fields (notifier, get, and set methods).

14 Risks and Technical Debt

This chapter lists and rates risks associated with the overall architecture of the AUTOSAR Adaptive Platform in Section 14.1. These risks usually might cause that some of the quality attributes of the AUTOSAR Adaptive Platform are not (fully) met. Section 14.2 lists technical debt of the AUTOSAR Adaptive Platform that may impact its maintainability.

14.1 Risks

14.1.1 Risk Assessment

This document categorizes risks according to their severity. The severity is a function of the probability and the impact of a risk. The probabilities are categorized as follows:

- **very low** - probability is less than 1 thousandth
- **low** - probability is between 1 thousandth and 1 percent
- **medium** - probability is between 1 percent and 10 percent
- **high** - probability is between 10 percent and 50 percent
- **very high** - probability is more than 50 percent

The impact of a risk is categorized as follows:

- **very low** - at most one quality scenario will take additional significant effort to be satisfied
- **low** - more than one quality scenario will take additional significant effort to be satisfied
- **medium** - at most one quality scenario is not satisfied with small gaps
- **high** - at most one quality scenario is not satisfied with big gaps
- **very high** - more than one quality scenario is not satisfied with big gaps

The final severity of a risk is then calculated according to table 14.1.

Impact	Probability				
	very low	low	medium	high	very high
very low	low (1)	low (2)	low (3)	medium (4)	medium (5)
low	low (2)	medium (4)	medium (6)	high (8)	high (10)
medium	low (3)	medium (6)	high (9)	high (12)	high (15)
high	medium (4)	high (8)	high (12)	extreme (16)	extreme (20)
very high	medium (5)	high (10)	high (15)	extreme (20)	extreme (25)

Table 14.1: Risk Severity Matrix

14.1.2 Risk List

No architectural risks were identified yet.

14.2 Technical Debt

No technical debt has been identified yet.

References

- [1] ISO 42010:2011 – Systems and software engineering – Architecture description
<http://www.iso.org>
- [2] Explanation of Adaptive and Classic Platform Software Architectural Decisions
AUTOSAR_EXP_SWArchitecturalDecisions
- [3] Glossary
AUTOSAR_TR_Glossary
- [4] Main Requirements
AUTOSAR_RS_Main
- [5] General Requirements specific to Adaptive Platform
AUTOSAR_RS_General
- [6] ATAMSM: Method for Architecture Evaluation
https://resources.sei.cmu.edu/asset_files/TechnicalReport/2000_005_001_13706.pdf
- [7] Agile Software Development: Principles, Patterns, and Practices
- [8] Guide to the Software Engineering Body of Knowledge, Version 3.0
www.swebok.org
- [9] Specification of Manifest
AUTOSAR_TPS_ManifestSpecification
- [10] Specification of SW-C End-to-End Communication Protection Library
AUTOSAR_SWS_E2ELibrary